

Waft Larval Form: Complete Technical Specification v0.6.0

Summary: Complete Specification: All aspects of the system documented Migration-Ready Design: Database schema compatible with future Redbean form Error Resilience: Novel approach to error...

Generated: 2026-01-12 23:46 UTC | Ideas Extracted: 40

What Happened

Complete Specification: All aspects of the system documented Migration-Ready Design: Database schema compatible with future Redbean form Error Resilience: Novel approach to error handling through TRAUMA logging Reactive Architecture: Efficient change detection and automatic UI updates Comprehensive Export: Multiple format support for data analysis and backup.

This document presents a complete technical specification for the Waft Larval Form, a Python-based application serving as the developmental stage preceding the Redbean "Mature Form" implementation. The system implements a 3D printing workflow manager that tracks G-code files and print jobs, built on a philosophical foundation termed "Hasvanism" (Breath, Memory, Trauma). The architecture demonstrates a novel approach to error resilience, where errors are not ignored but rather etched into persistent memory as "TRAUMA" events, allowing the system to continue operating while maintaining a complete historical record. The implementation utilizes Python 3.x with Streamlit for the user interface, SQLite 3 for persistent storage, and Pandas for data processing. The system features a reactive live reload mechanism, comprehensive data export capabilities, and a migration-ready database schema designed for seamless transition to the mature Redbean form. This specification provides complete implementation details including database schema, core classes, user interface components, error handling strategies, and testing requirements.

The Waft Larval Form represents a transitional stage in the evolution of the Waft system architecture, designed as a developmental precursor to the Redbean "Mature Form". This application serves as a 3D printing workflow manager that tracks G-code files and print jobs, implementing a unique philosophical framework that guides both its design and operational behavior.

Additional Details

This philosophical foundation creates an entity that "breathes" (executes logic), "remembers" (persists to database), and "feels pain" (logs errors as TRAUMA). Unlike traditional error handling approaches that may suppress or ignore errors, this system maintains a complete chronicle of all events, including failures, ensuring that the system's history is preserved and can be analyzed.

waftlarva.py (Single-file application) |— Configuration (Constants, Enums) |— Database Connection (*getdb_connection* with retry logic) |— WaftEntity Class (Core consciousness) | |— Database initialization | |— Chronicle logging | |— Artifact management | |— Data export methods | |— Data hash for change detection |— Streamlit UI (main function) |— Header with auto-refresh controls |— Status summary dashboard |— Activity log panel |— Print job management panel |— Data export section |— Reactive update system.

[x] Database schema (chronicle, artifacts) [x] WaftEntity class with all methods [x] Chronicle logging system [x] Artifact management [x] Safe breath error handling [x] Database connection with retry logic [x] WAL mode for concurrency.

Content Breakdown

Type	Count
Decisions	3
Insights	3
Actions	17
Concepts	17
Questions	0